Introdução à Computação
tries e hash tables

Alexandre Rademaker

trie I

Trie (nome derivado de 'retrieval') é um tipo de árvore de pesquisa k-ária usada para armazenar e pesquisar uma chave específica de um conjunto. Usando Trie, as complexidades de pesquisa pode ser limitada ao (comprimento da maior chave).

Também chamadas árvores de prefixos, uma vez que prefixos comuns compartilham os mesmos ramos (branches).

Podem ser usadas para armazenar um arrays associativos em que as chaves são normalmente cadeias de caracteres.

trie II

Imagine armazenarmos strings (chaves) em uma árvore binária de busca balanceada.

Para localizar uma chave, precisariamos de um tempo proporcional a $M \log N$, onde M é o comprimento máximo das chaves e N é o número de chaves na árvore.

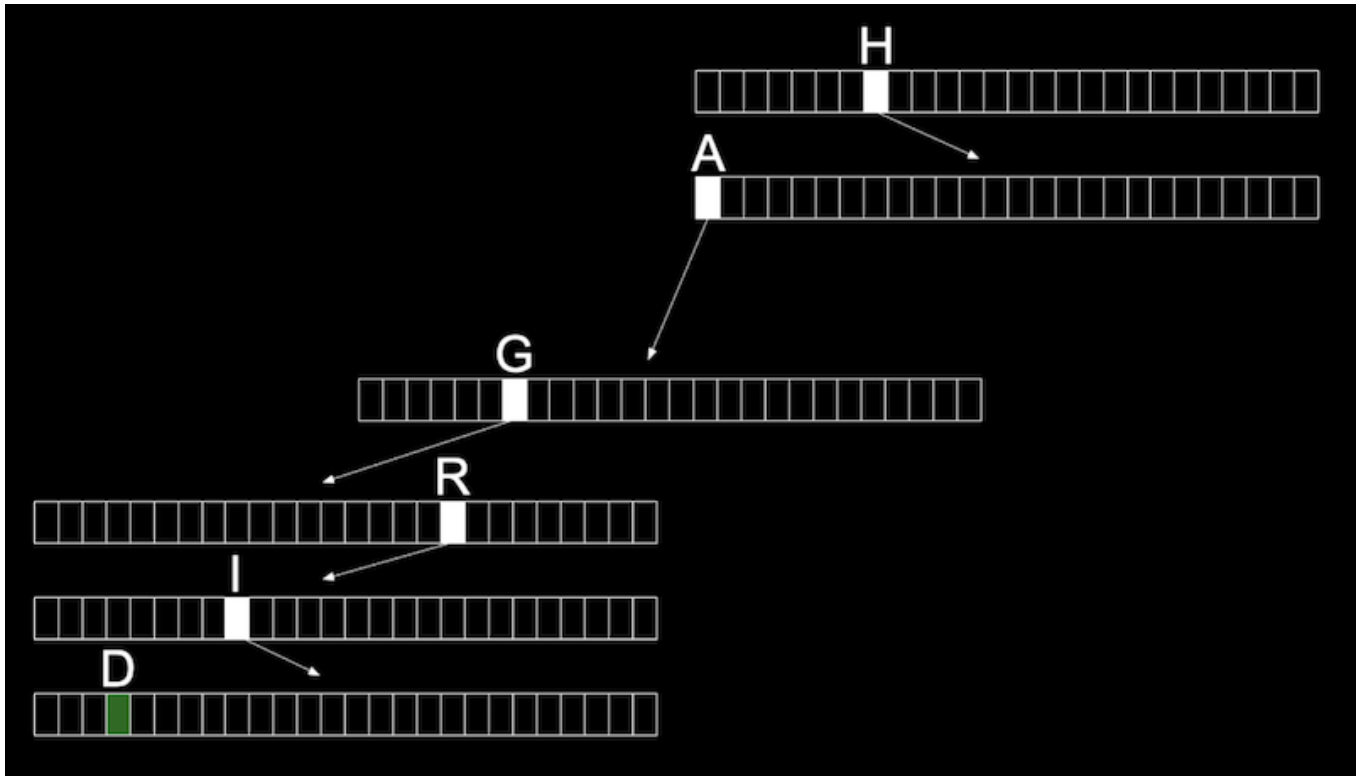
Usando Trie, a chave pode ser pesquisada em tempo O(M). No entanto, a penalidade está nos requisitos de armazenamento do Trie.

trie III

Uma trie é uma árvore onde os nós armazenam algum valor e um array de ponteiros para outros nós, uma posição para cada elemento do alfabeto das chaves.

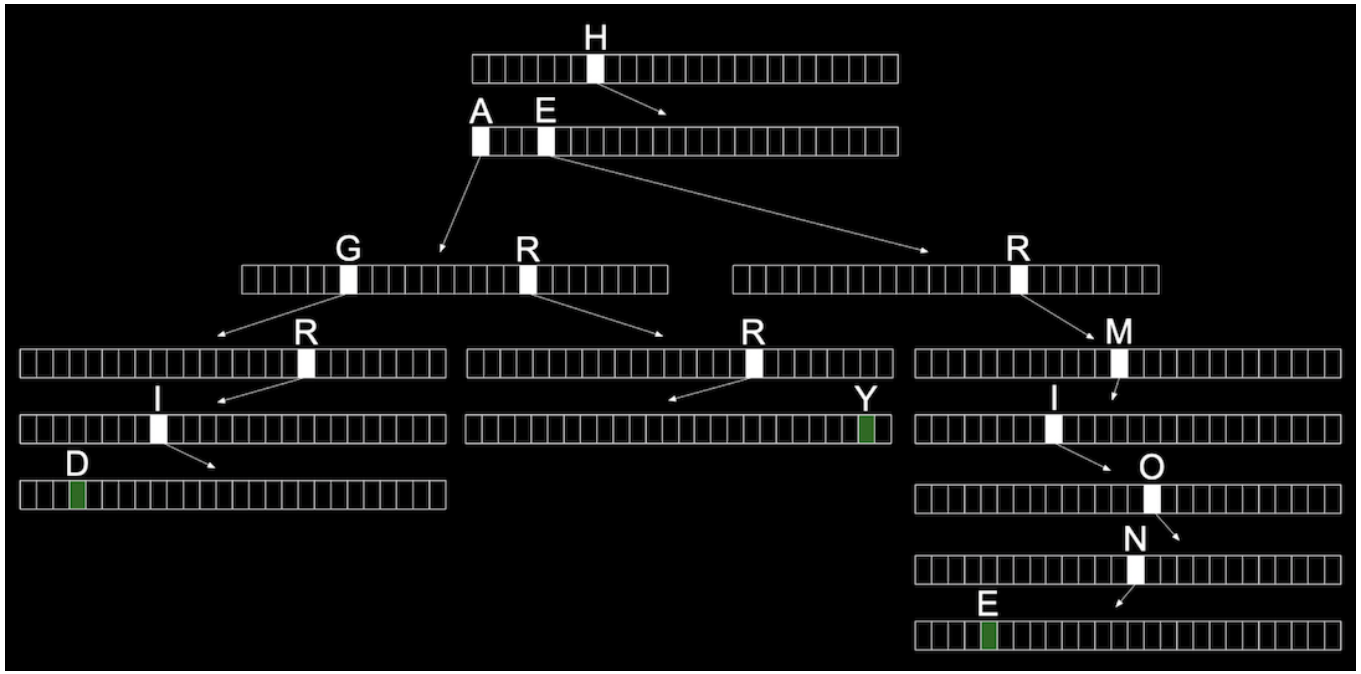
trie IV

Uma trie com a palavra ‘hagrid’ ficaria



trie V

A medida que mais nomes forem armazenados, mais reuso de espaço é possível: ‘hagrid’, ‘harry’ e ‘hermione’.



trie em C I

Podemos definir uma **trie** em C com

```
1 typedef struct node
2 {
3   bool is_word;
4   struct node *children[SIZE_OF_ALPHABET];
5 } t_node;
```

Valor `bool` indica se este nó indica final de uma palavra armazenada. Mas poderíamos armazenar outros valores associados a chave que representa o prefixo até o dado nó.

O array de ponteiros para os próximos nós 'filhos', próximo caractere da chave.

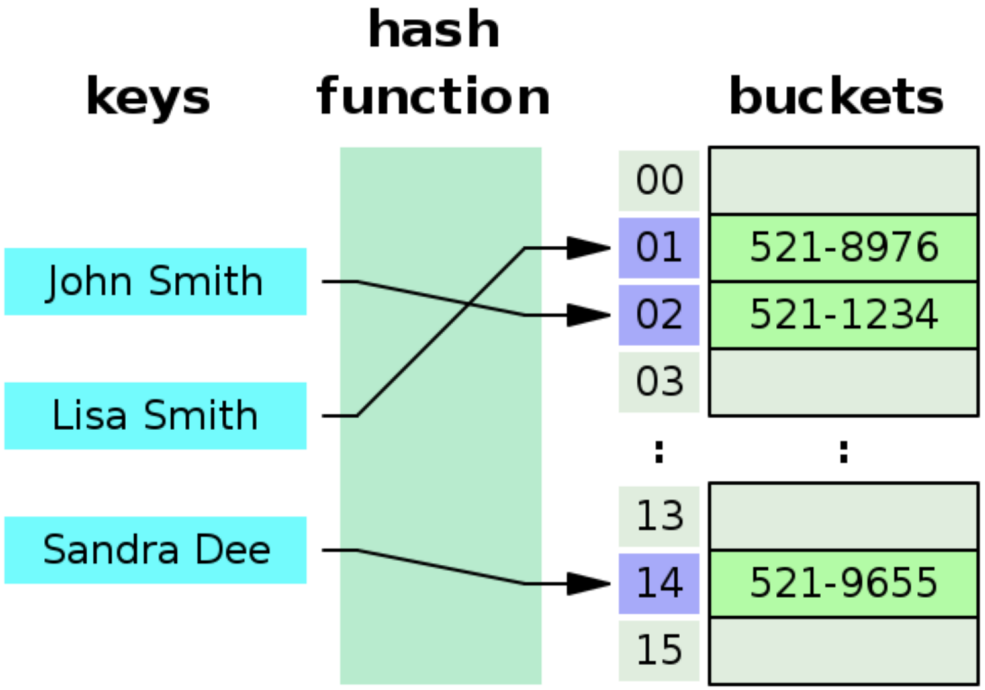
trie em C II

A altura da **trie** é o tamanho da maior palavra armazenada.

Independente do número de palavras, o tempo de busca de uma palavra será o tamanho desta palavra.

Mas muitos espaços estarão desocupados com `null`.

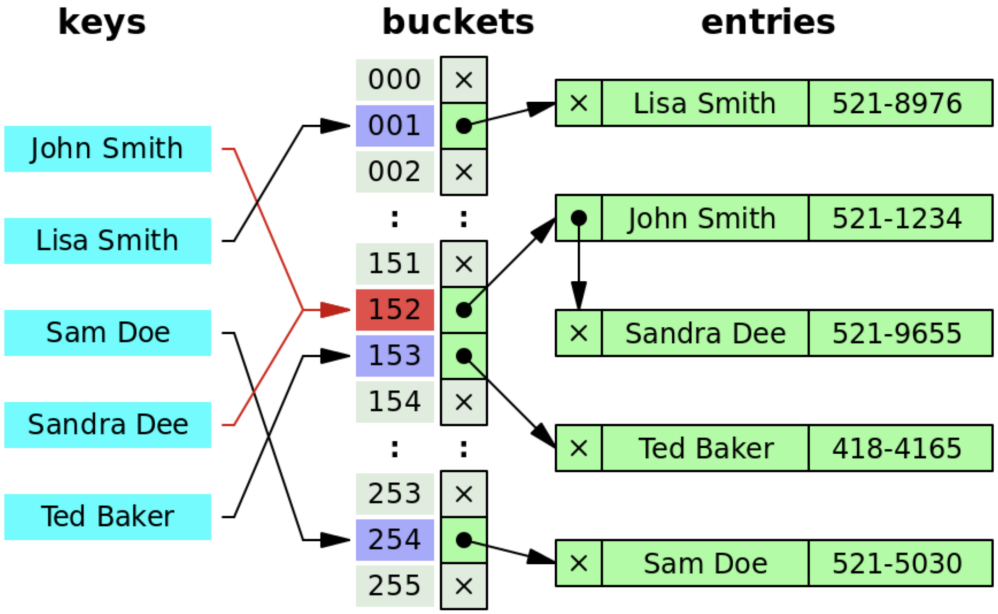
hash table I



https://en.wikipedia.org/wiki/Hash_table

hash table II

cada chave tem apenas um valor, mas colisões podem ocorrer



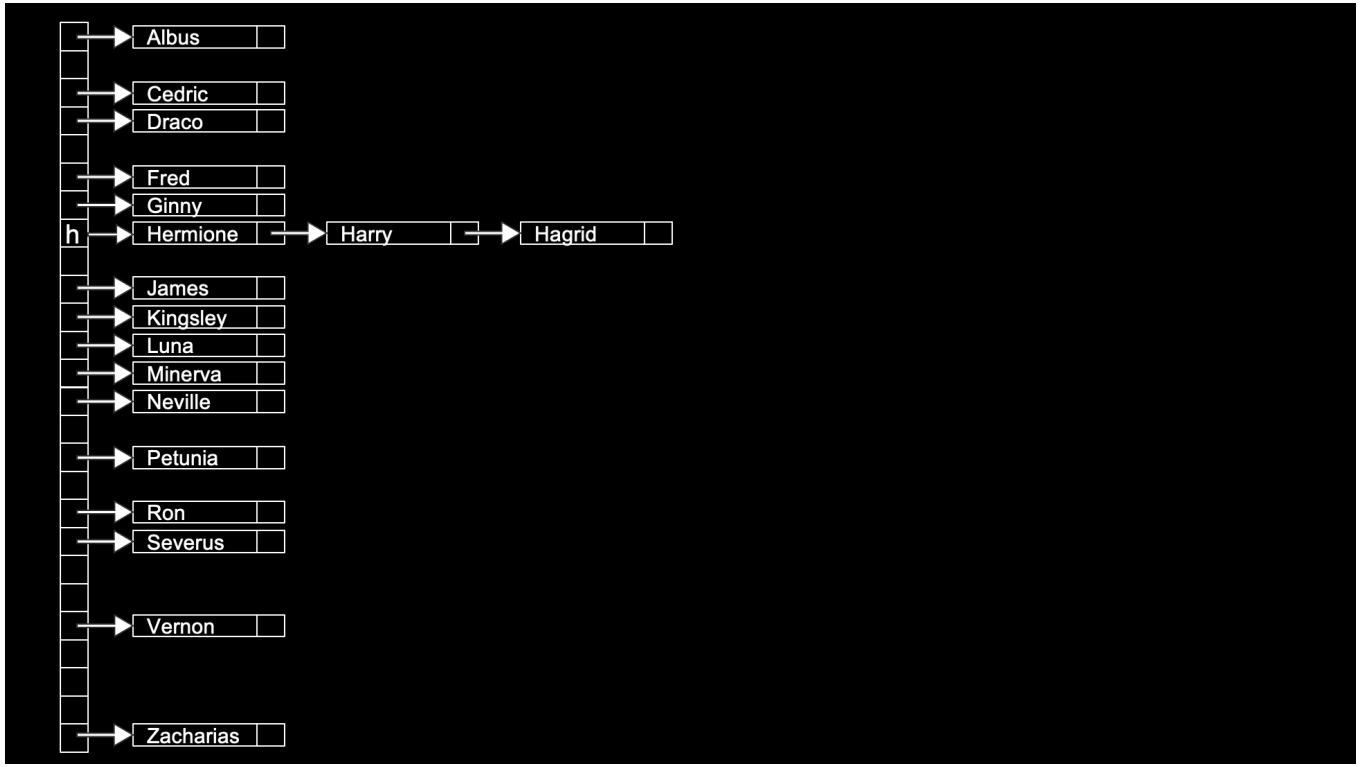
https://en.wikipedia.org/wiki/Hash_table

hash table III

Podemos pensar nas chaves como as letras A até Z, e inserir nomes de acordo com as iniciais.

hash table IV

E os conflitos quando tivermos que armazenar nomes com mesmas iniciais.



hash table V

Cada nós terá um array de caracteres pré-allocado, de tamanho máximo `LONGEST_WORD + 1` para armazenar o nome sendo guardado. O ponteiro `next` aponta para outro nó, se houver conflito.

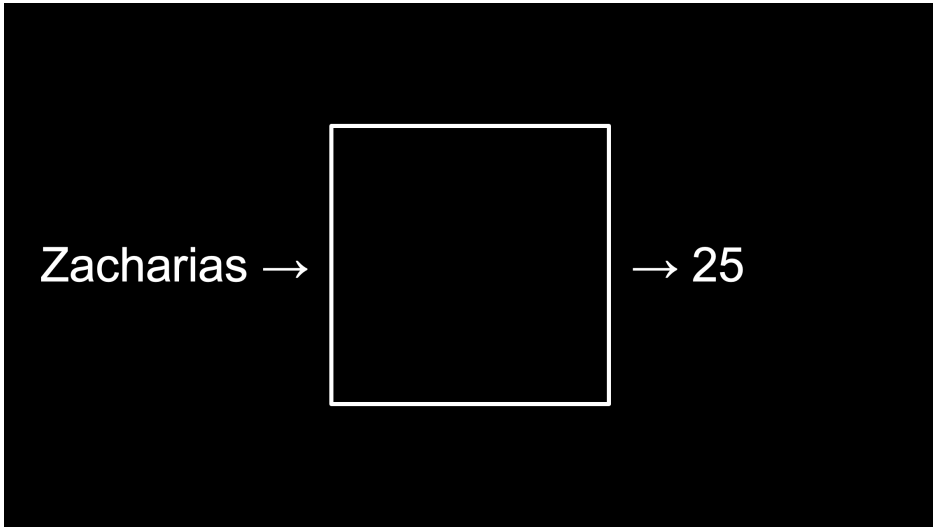
```
1 typedef struct node
2 {
3   char word[LONGEST_WORD + 1];
4   struct node *next;
5 }
6 node;
```

Um array de ponteiros para `node`s com um número de 'lugares' pré-definido, como 26, no exemplo acima (A-Z).

```
1 node *hash_table[NUMBER_OF_BUCKETS];
```

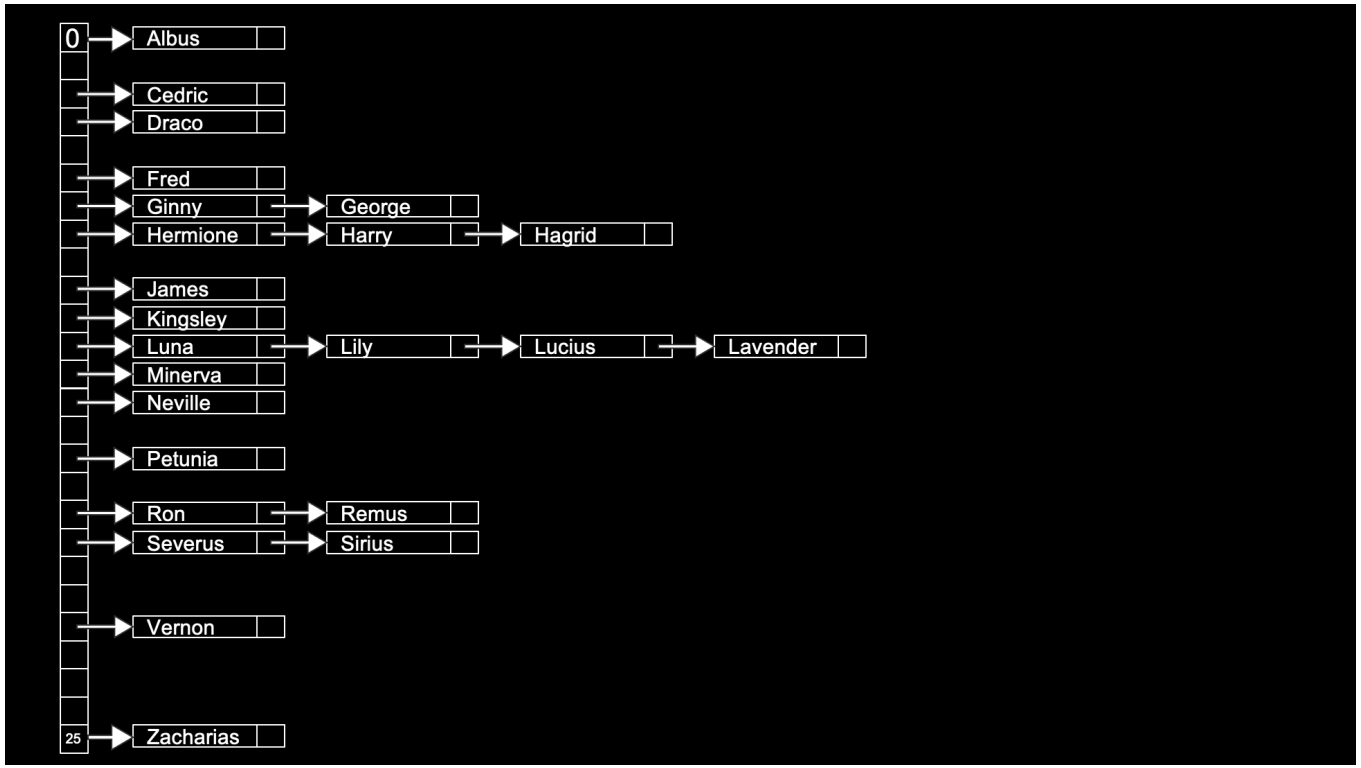
hash table VI

Lembrando que a função hash mapea cada nome para uma localização possível da tabela hash, neste caso uma letra A-Z (ou seu código ASCII)



hash table VII

Os endereços na tabela hash, se associados aos códigos ASCII, poderiam ser traduzidos em posições diretamente.



hash table VIII

Para reduzir os conflitos (e tamanho das sequências lineares de nós), poderíamos mudar as chaves para as **duas letras iniciais**

agora teríamos 676 posições na tabela, para todas as combinações das 2 primeiras letras.

hash table IX

Podemos tentar aumentar o 'buckets' para 26^3 (3 iniciais)

reduzindo conflito mais aumentando possíveis endereços não ocupados. Pior caso de busca são todos os valores em uma mesma posição, $O(n)$, busca na lista encadeada.